Unit – 3

Introduction to JavaScript:-

JavaScript is a fundamental web technology that allows developers to create dynamic and interactive web pages. This is primarily achieved through the use of functions, events, and the manipulation of the Document Object Model (DOM).

JavaScript Functions

A JavaScript function is a reusable block of code designed to perform a specific task. Functions help organize and modularize code, making it more manageable.

- **Definition:** Functions are defined using the `function` keyword, followed by a name, parentheses (which may contain parameters), and curly braces {} that enclose the function's statements.
- **Invocation:** A function's code is executed only when it is called or "invoked".
- **Parameters and Arguments:** Functions can accept input values called parameters, and when the function is called, these inputs are provided as arguments.
- **Return Values:** Functions can process data and return an output value using the `return` statement.

  Example:

```
function greetUser(name) { // 'name' is a parameter
  return "Hello, " + name + "!"; // Returns an output value
}

// Invoking the function with an argument
let message = greetUser("Alice");
console.log(message); // Output: Hello, Alice!
```

JavaScript Events

Events are actions or occurrences that happen in the browser, such as a user clicking a button, pressing a key, or the page finishing loading. JavaScript's interaction with HTML is handled through events, which trigger specific functions (event handlers).

Common events include:

- `click`: When a user clicks an element.
- `mouseover`: When the mouse pointer moves over an element.
- `keydown`: When a user presses a key on the keyboard.

Example:

```
// A function to be executed when an event occurs
function changeText() {
  document.getElementById("myElement").innerHTML = "Text Changed!";
}
// Attaching the 'click' event listener to a button
document.getElementById("myButton").addEventListener("click", changeText);
```

Document Object Model (DOM) Traversing

The DOM is a programming interface for web documents that represents the page's structure as a tree of objects (nodes). This tree structure includes elements, text, and attributes. DOM traversing refers to the process of navigating this tree using JavaScript to access and manipulate elements.

- **Accessing Elements:**
  - `document.getElementById("idName")`: Selects an element by its unique ID.
  - `document.querySelector(".className")`: Selects the first element that matches a specified CSS selector.
  - `document.getElementsByTagName("tagName")`: Returns a collection of elements with a given tag name.
- **Navigating the Hierarchy (relative to a specific node):**
  - `parentNode`: Accesses the immediate parent node.
  - `childNodes`: Returns a collection of all child nodes (including text and comment nodes).
  - `firstChild` / `lastChild`: Accesses the first or last child node.
  - `nextSibling` / `previousSibling`: Accesses sibling nodes at the same level of the tree.
  - `children`: (A more modern and cleaner alternative to `childNodes`) returns only element nodes, filtering out text/comment nodes.

Output System in JavaScript

In JavaScript, the Output System refers to the collection of methods used to display data or communicate information. Since JavaScript doesn't have a built-in "print" function that connects directly to a printer or a screen, it relies on the environment (the Browser or Node.js) to provide "output interfaces."

Think of the output system as a set of different "speakers." Depending on who needs to hear the information—the user, the developer, or the HTML page itself—you choose a different speaker.

1. What is an Alert ?

An alert is a **modal dialog box**. "Modal" means that it takes complete control of the browser. When an alert pops up, the user cannot click anything else on the page, and the JavaScript code **pauses** entirely until the "OK" button is clicked.

Basic Syntax:

```
window.alert("Hello! This is an alert box.");
// Or simply:
alert("Hello! This is an alert box.");
```

❖ **Key Characteristics:**

- **Blocking Nature:** It is a "blocking" function. If you have a loop running or a timer, everything freezes while the alert is visible.
- **Simple Interface:** It only provides a message and an **"OK"** button. You cannot add custom buttons like "Cancel" or "Close" to a standard alert.
- **Plain Text:** It does not render HTML. If you try to pass Hello, it will literally display the tags as text rather than making the word bold.

❖ **The "Alert Family" (Related Methods):**

The alert() is part of a trio of dialog methods used for different levels of interaction:

Method	Purpose	Return Value
alert()	To give information to the user.	undefined
confirm()	To ask a Yes/No question.	true or false
prompt()	To ask for text input.	The string entered or null

❖ **Pros and Cons:**

The Good :

- **Guaranteed Visibility:** The user literally cannot ignore it.
- **Zero Setup:** You don't need to create any HTML or CSS; it’s built into the browser.
- **Debugging:** It’s a quick (though old-school) way to check if a specific line of code is being reached.

The Bad

- **Poor User Experience:** Most modern users find pop-up alerts annoying and intrusive.
- **Ugly Design:** You cannot style an alert box with CSS. It looks different in Chrome than it does in Safari or Firefox.
- **Abuse Prevention:** Modern browsers will offer users a checkbox to "Prevent this page from creating additional dialogs" if alerts happen too frequently.

---

2. What is Throughput?

Throughput is the rate at which a system processes requests or outputs data. In JavaScript (especially in Node.js or high-performance web apps), it is often measured in:

- Requests Per Second (RPS)
- Frames Per Second (FPS) (for UI output/animations)
- Megabytes per second (MB/s) (for data streams)

❖ **Throughput vs. Latency**

These two terms are often confused but represent different parts of the output system:

- **Latency:** The time it takes for a *single* output to happen (e.g., how long after a click does the alert() appear?)**.**
- **Throughput:** How many total outputs can happen in a minute (e.g., how many logs can the console handle before the browser lags?).

❖ **How Different Output Methods Affect Throughput.**

Different output types have a massive impact on your application's "bandwidth" or speed:

Low Throughput: window.alert()

- **The Bottleneck:** Because alert() is **blocking**, its throughput is effectively **zero** while the box is open.
- **Result:** The entire engine stops. You cannot process more data until the user interacts.

Medium Throughput: console.log()

- **The Bottleneck:** Writing thousands of lines to the console takes CPU power and memory.
- **Result:** If you log inside a heavy loop, your app's throughput will drop significantly because the browser is busy formatting text for the dev tools.

High Throughput: innerHTML / Virtual DOM

- **The Solution:** Modern frameworks (like React) optimize throughput by batching updates. Instead of updating the screen 1,000 times, they wait and do it once.
- **Result:** High FPS and smooth performance.

❖ **Measuring Throughput in JS**

You can measure the throughput of a specific code block using performance.now():

```
let start = performance.now();
let count = 0;

while (performance.now() - start < 1000) { // Run for 1 second
// Perform an output operation
// document.getElementById('output').textContent = count;
count++;
}

console.log(`Throughput: ${count} operations per second.`);
```

3.Input box:

In JavaScript, an **Input Box** is the primary bridge used to move data from the user into your code's logic. It is the "I" in the **IPO (Input-Process-Output)** model.

There are two main ways to create an input box: the built-in browser dialog and the standard HTML element.

❖ **The Built-in Dialog: window.prompt()**

This is the simplest way to get input. It triggers a browser-generated pop-up window with a text field and "OK/Cancel" buttons.

- **Behavior:** It is **synchronous and blocking**. This means the entire JavaScript engine stops, and the user cannot interact with the rest of the page until they close the box.
- **Throughput: Zero.** Because it halts execution, it has the lowest possible throughput of any input method.

Code Example:

```
let favoriteColor = prompt("What is your favorite color?", "Blue");
console.log(favoriteColor);
```

❖ **The Standard UI Element: HTML <input>**

This is the professional method used in modern web applications. The input box lives directly on the webpage.

- **Behavior:** It is **non-blocking**. The user can type while other scripts, animations, and data fetches continue in the background.
- **Throughput: High.** Your code can listen for every single keystroke (oninput) or wait for a button click without ever stopping the program.
- **The Process:**
    1. **Define in HTML:** <input type="text" id="userIn">
    2. **Access in JS:** Use the .value property.

```
// Reading the value when a button is clicked
function handleInput() {
   let data = document.getElementById("userIn").value;
   alert("You typed: " + data);
}
```

❖ **Input Box "Traps" & Tips**

A. Data Type Conversion

Every input box (both prompt and HTML <input>) returns data as a **String**. If you want to do math, you must convert the output.

- **The Problem:** "10" + "5" = "105"
- **The Fix:** Number("10") + Number("5") = 15

B. Validation

Since you can't control what a user types, you should always validate the input before processing it.

- Check if it is empty: if (input === "") { ... }
- Check if it is a number: isNaN(input).

4.Console

In the JavaScript output system, the **Console** is the most powerful tool for developers. It is a non-persistent, non-blocking environment used to inspect data, debug logic, and monitor the "health" of an application.

Unlike alert(), which interrupts the user, the console remains hidden unless the developer manually opens it (usually by pressing **F12** or **Right-Click > Inspect**).

❖ **The console Object**

The console is not just for simple text; it is an object with various methods designed for different types of data output.

A. console.log()

The standard method for outputting general information. It can handle strings, numbers, objects, and arrays.

- **Usage:** console.log("User logged in at:", new Date());

B. console.error() and console.warn()

Used to categorize output. Browsers typically highlight errors in **red** and warnings in **yellow**.

- **Usage:** console.error("API Connection Failed");

C. console.table()

A high-throughput method for visualizing data. It takes an array of objects and displays them in a clean, sortable grid.

- **Usage:** console.table([{name: "Alice", id: 1}, {name: "Bob", id: 2}]);

❖ **Console and Throughput**

The console is a **high-throughput** system, but it is not "free."

- **Non-Blocking:** Unlike alert(), console.log() does not stop the execution of your code. Your loops and animations continue to run while the console updates.
- **The "Observer" Effect:** If you attempt to log 10,000 items in a single second inside a loop, the browser's performance will drop. The overhead of formatting and rendering text in the DevTools window can eventually bottleneck your application's throughput.

❖ **. Input via Console**

The console is also a **Read-Eval-Print Loop (REPL)**. This means you can use it as an **Input Box** for testing:

1. You can type variables or functions directly into the console.
2. Press **Enter**.
3. JavaScript executes that code immediately and outputs the result.

Variables

Variables in JavaScript are used to store data values. They can be declared in different ways depending on how the value should behave.

- Variables can be declared using var, let, or const.
- JavaScript is dynamically typed, so types are decided at runtime.
- You don’t need to specify a data type when creating a variable.

❖ **The Three Ways to Define Variables**

JavaScript uses three specific keywords to define (or "declare") a variable. Each handles the system's memory and **throughput** differently.

A. const (Constant):

Use this for values that **will not change**. It is the most "stable" way to store data because the system knows exactly what to expect.

- **Example:** const pi = 3.14;
- **Rule:** You cannot re-assign it. Trying to change it will throw an error in the **Console**.

B. let (Block-Scoped)

The modern standard for values that **will change**.

- **Example:** let score = 0;
- **Rule:** Use this for counters, inputs from a user, or any data that evolves during the program.

C. var (Legacy)

The original way to define variables.

- **Warning:** In modern development, var is generally avoided because it has "global" tendencies that can cause bugs in complex output systems.

❖ **. The Anatomy of a Variable**

To define a variable, you follow a specific syntax:

1. **Declaration:** The keyword (let, const).
2. **Assignment:** The name (identifier) and the value.

Keyword + Name = Value

```
let userAge = 25;
// 'userAge' is the label for the memory location holding the number 25.
```

❖ **Rules for Naming Variables**

When naming variables in JavaScript, follow these rules

- Variable names must begin with a letter, underscore (_), or dollar sign ($).
- Subsequent characters can be letters, numbers, underscores, or dollar signs.
- Variable names are case-sensitive (e.g., age and Age are different variables).
- Reserved keywords (like function, class, return, etc.) cannot be used as variable names.

❖ **Declaring Variables**

To use a variable, you must first "declare" it. JavaScript provides three keywords to do this, each with different rules regarding how the data can be used or changed.

Keyword	**Scope**	**Reassignable?**	**Redeclarable?**
let	Block {}	**Yes**	No
const	Block {}	**No**	No
var	Function	**Yes**	Yes

1. let

Introduced in modern JavaScript (ES6), let is the standard way to declare variables that you expect to change later. It is **block-scoped**, meaning it only exists within the curly braces {} where it was defined.

```
let score = 10;
score = 15; // This works!
```

2. const:

Short for "constant," this is used for values that should **not** be reassigned. Use this by default unless you know the value needs to change; it makes your code more predictable.

```
const pi = 3.14159;
// pi = 4; // This would throw an error!
```

3. var

The "old school" way to declare variables. It is **function-scoped**, which can lead to bugs because it doesn't respect block boundaries (like if statements or for loops). Most modern developers avoid var entirely.

```
var a = "Hello Geeks";
var b = 10;
console.log(a);
console.log(b);
```

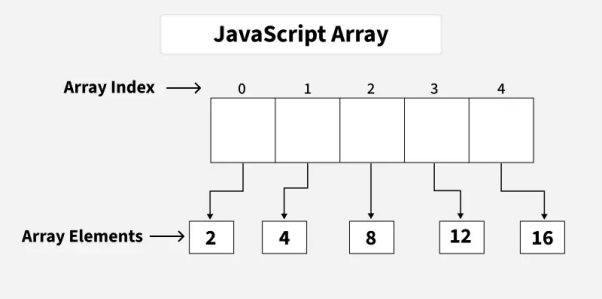
❖ **The Life Cycle of a Variable**

1. **Declaration:** Telling the computer the name exists (e.g., let username;).
2. **Initialization:** Giving the variable its first value (e.g., username = "Alex";).
3. **Usage:** Accessing that value later in your code.

JavaScript Arrays

In JavaScript, an array is an ordered list of values. Each value, known as an element, is assigned a numeric position in the array called its index. The indexing starts at 0, so the first element is at position 0, the second at position 1, and so on.

Arrays can hold any type of data-such as numbers, strings, objects, or even other arrays—making them a flexible and essential part of JavaScript programming.



Example:

If you have a list of items (a list of car names, for example), storing the cars in single variables could look like this:

```
let car1 = "Saab"
let car2 = "Volvo"
let car3 ="BMW"
```

1. Create Array using Literal

Creating an array using array literal involves using square brackets [] to define and initialize the array.

```
// Creating an Empty Array

let a = [];

console.log(a);

// Creating an Array and Initializing with Values

let b = [10, 20, 30];

console.log(b);
```

Output

```
[]

[ 10, 20, 30 ]
```

2. Create using new Keyword (Constructor)

The "**Array Constructor**" refers to a method of creating arrays by invoking the Array constructor function.

```
// Creating and Initializing an array with values

let a = new Array(10, 20, 30);

console.log(a);
```

Output

```
[ 10, 20, 30 ]
```

Note: *Both the above methods do exactly the same. Use the array literal method for efficiency, readability, and speed.*

Basic Operations on JavaScript Arrays

1. Accessing Elements of an Array

Any element in the array can be accessed using the index number. The index in the arrays starts with 0.

```
// Creating an Array and Initializing with Values

let a = ["HTML", "CSS", "JS"];

// Accessing Array Elements
```

```
console.log(a[0]);

console.log(a[1]);
```

Output

```
HTML

CSS
```

2. Accessing the First Element of an Array

The array indexing starts from 0, so we can access first element of array using the index number.

```
// Creating an Array and Initializing with Values

let a = ["HTML", "CSS", "JS"];

// Accessing First Array Elements

let fst = a[0];

console.log("First Item: ", fst);
```

Output

```
First Item:  HTML
```

3. Accessing the Last Element of an Array

We can access the last array element using [array.length - 1] index number.

```
// Creating an Array and Initializing with Values

let a = ["HTML", "CSS", "JS"];

// Accessing Last Array Elements

let lst = a[a.length - 1];

console.log("First Item: ", lst);
```

Output

```
First Item:  JS
```

4. Modifying the Array Elements

Elements in an array can be modified by assigning a new value to their corresponding index.

```
// Creating an Array and Initializing with Values

let a = ["HTML", "CSS", "JS"];

console.log(a);

a[1]= "Bootstrap";

console.log(a);
```

Output

```
[ 'HTML', 'CSS', 'JS' ]

[ 'HTML', 'Bootstrap', 'JS' ]
```

5. Adding Elements to the Array

Elements can be added to the array using methods like push() and unshift().

- The push() method add the element to the end of the array.
- The unshift() method add the element to the starting of the array.

```
// Creating an Array and Initializing with Values
let a = ["HTML", "CSS", "JS"];
// Add Element to the end of Array
a.push("Node.js");
// Add Element to the beginning
a.unshift("Web Development");
console.log(a);
```

Output

```
[ 'Web Development', 'HTML', 'CSS', 'JS', 'Node.js' ]
```

6. Removing Elements from an Array

To remove the elements from an array we have different methods like pop(), shift(), or splice().

- The pop() method removes an element from the last index of the array.
- The shift() method removes the element from the first index of the array.
- The splice() method removes or replaces the element from the array.

```
// Creating an Array and Initializing with Values
let a = ["HTML", "CSS", "JS"];
console.log("Original Array: " + a);
// Removes and returns the last element
let lst = a.pop();
console.log("After Removing the last: " + a);
// Removes and returns the first element
let fst = a.shift();
console.log("After Removing the First: " + a);
// Removes 2 elements starting from index 1
a.splice(1, 2);
console.log("After Removing 2 elements starting from index 1: " + a);
```

Output

```
Original Array: HTML,CSS,JS
After Removing the last: HTML,CSS
After Removing the First: CSS
After Removing 2 elements starting from index 1: CSS
```

7. Array Length

We can get the length of the array using the array length property.

```
// Creating an Array and Initializing with Values

let a = ["HTML", "CSS", "JS"];

let len = a.length;

console.log("Array Length: " + len);
```

Output

```
Array Length: 3
```

8. Increase and Decrease the Array Length

We can increase and decrease the array length using the JavaScript length property.

```
// Creating an Array and Initializing with Values

let a = ["HTML", "CSS", "JS"]

// Increase the array length to 7

a.length = 7;

console.log("After Increasing Length: ", a);

// Decrease the array length to 2

a.length = 2;

console.log("After Decreasing Length: ", a)
```

Output

```
After Increasing Length:  [ 'HTML', 'CSS', 'JS', <4 empty items> ]

After Decreasing Length:  [ 'HTML', 'CSS' ]
```

9. Iterating Through Array Elements

We can iterate array and access array elements using for loop and forEach loop.

Example: It is an example of for loop.

```
// Creating an Array and Initializing with Values

let a = ["HTML", "CSS", "JS"];

// Iterating through for loop

for (let i = 0; i < a.length; i++) {

   console.log(a[i])

}
```

Output

```
HTML

CSS

JS
```

Example: It is the example of Array.forEach() loop.

```
// Creating an Array and Initializing with Values
```

```
let a = ["HTML", "CSS", "JS"];

// Iterating through forEach loop

a.forEach(function myfunc(x) {

   console.log(x);

});
```

Output

```
HTML

CSS

JS
```

10. Array Concatenation

Combine two or more arrays using the concat() method. It returns new array containing joined arrays elements.

```
// Creating an Array and Initializing with Values

let a = ["HTML", "CSS", "JS", "React"];

let b = ["Node.js", "Expess.js"];

// Concatenate both arrays

let concateArray = a.concat(b);

console.log("Concatenated Array: ", concateArray);
```

Output

```
Concatenated Array:  [ 'HTML', 'CSS', 'JS', 'React', 'Node.js', 'Expess.js' ]
```

11. Conversion of an Array to String

We have a builtin method **toString()** to converts an array to a string.

```
// Creating an Array and Initializing with Values

let a = ["HTML", "CSS", "JS"];

// Convert array ot String

console.log(a.toString());
```

Output

```
HTML,CSS,JS
```

12. Check the Type of an Arrays

The JavaScript typeof operator is used ot check the type of an array. It returns "object" for arrays.

```
// Creating an Array and Initializing with Values

let a = ["HTML", "CSS", "JS"];

// Check type of array

console.log(typeof a);
```

Output

object

Recognizing a JavaScript Array

There are two methods by which we can recognize a JavaScript array:

- **By using Array.isArray() method**
- **By using instanceof method**

Below is an example showing both approaches:

```
const courses = ["HTML", "CSS", "Javascript"];

console.log("Using Array.isArray() method: ", Array.isArray(courses))

console.log("Using instanceof method: ", courses instanceof Array)
```

Output

```
Using Array.isArray() method:  true

Using instanceof method:  true
```

Note: A common error is faced while writing the arrays:

```
const a = [5]
// and
const a = new Array(5)
```

The above two statements are not the same.

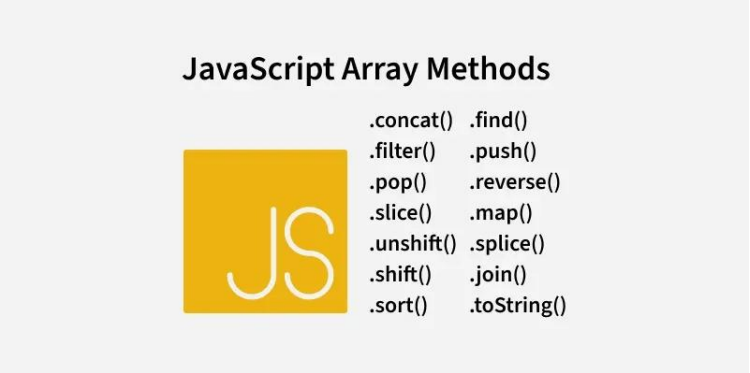
Output: This statement creates an array with an element " [5] ". Try with the following examples

```
// Example 1

const a1 = [5]

console.log(a1)

// Example 2

const a2 = new Array(5)

console.log(a2)
```

Output

```
[ 5 ]

[ <5 empty items> ]
```

JavaScript Array Methods



1. JavaScript Array length

The length property of an array returns the number of elements in the array. It automatically updates as elements are added or removed.

```
let a = ["HTML", "CSS", "JS", "React"];

console.log(a.length);
```

Output

4

In this example

- The code defines an array 'a' containing the elements "HTML", "CSS", "JS", and "React".
- a.length returns the number of elements in the array.

2. JavaScript Array toString() Method

The toString() method converts the given value into the string with each element separated by commas.

```
let a  = ["HTML", "CSS", "JS", "React"];

let s = a.toString();

console.log(s);
```

Output

HTML,CSS,JS,React

In this example

- The code defines an array "a" containing the elements "HTML", "CSS", "JS", and "React".
- The toString() method converts the array 'a' into a string.

3. JavaScript Array join() Method

This join() method creates and returns a new string by concatenating all elements of an array. It uses a specified separator between each element in the resulting string.

```
let a = ["HTML", "CSS", "JS", "React"];

console.log(a.join('|'));
```

Output

HTML|CSS|JS|React

In this example

- The code defines an array 'a' with the elements "HTML", "CSS", "JS", and "React".
- The join('|') method combines the array elements into a single string, with each element separated by a pipe (|) character.

4. JavaScript Array delete Operator

The delete operator is used to delete the given value which can be an object, array, or anything.

```
let emp = {
   firstName: "Riya",
   lastName: "Kaur",
   salary: 40000
}
console.log(delete emp.salary);
console.log(emp);
```

Output

```
true
{ firstName: 'Riya', lastName: 'Kaur' }
```

In this example

- The delete emp.salary statement removes the salary property from the emp object and returns true if successful.
- After deletion, console.log(emp) prints the updated object without the salary property.

5. JavaScript Array concat() Method

The concat() method is used to concatenate two or more arrays and it gives the merged array.

```
let a1 = [11, 12, 13];
let a2 = [14, 15, 16];
let a3 = [17, 18, 19];
let newArr = a1.concat(a2, a3);
console.log(newArr);
```

Output

```
[
  11, 12, 13, 14, 15,
  16, 17, 18, 19
]
```

In this example

- The code defines three arrays, a1, a2, and a3, and uses the concat() method to merge them into a single array newArr.

- The resulting array [11, 12, 13, 14, 15, 16, 17, 18, 19] is logged to the console, preserving the order of elements from the original arrays.

6. JavaScript Array flat() Method

The flat() method is used to flatten the array i.e. it merges all the given array and reduces all the nesting present in it.

```
const a1 = [['1', '2'], ['3', '4', '5',['6'], '7']];

const a2 = a1.flat(Infinity);

console.log(a2);
```

Output

```
[
  '1', '2', '3',
  '4', '5', '6',
  '7'
]
```

In this example

- The code defines a multilevel (nested) array 'a1' and uses the flat(Infinity) method to flatten it completely into a single-level array.

7. JavaScript Array.push() Method

The push() method is used to add an element at the end of an Array. As arrays in JavaScript are mutable objects, we can easily add or remove elements from the Array.

```
let a = [10, 20, 30, 40, 50];

a.push(60);

a.push(70, 80, 90);

console.log(a);
```

Output

```
[
  10, 20, 30, 40, 50,
  60, 70, 80, 90
]
```

8. JavaScript Array.unshift() Method

The unshift() method is used to add elements to the front of an Array.

```
let a = [20, 30, 40];

a.unshift(10, 20);

console.log(a);
```

Output

```
[ 10, 20, 20, 30, 40 ]
```

9. JavaScript Array.pop() Method

The pop() method is used to remove elements from the end of an array.

```
let a = [20, 30, 40, 50];

a.pop();

console.log(a);
```

Output

[20, 30, 40]

10. JavaScript Array.shift() Method

The shift() method is used to remove elements from the beginning of an array

```
let a = [20, 30, 40, 50];

a.shift();

console.log(a);
```

Output

[30, 40, 50]

11. JavaScript Array.splice() Method

The splice() method is used to Insert and Remove elements in between the Array.

```
let a = [20, 30, 40, 50];

a.splice(1, 3);

a.splice(1, 0, 3, 4, 5);

console.log(a);
```

Output

[20, 3, 4, 5]

- The first a.splice(1, 3) removes 3 elements (30, 40, 50) starting at index 1. The array becomes [20].
- The second a.splice(1, 0, 3, 4, 5) inserts 3, 4, and 5 at index 1 without removing anything. The array becomes [20, 3, 4, 5].

12. JavaScript Array.slice() Method

The slice() method returns a new array containing a portion of the original array, based on the start and end index provided as arguments

```
const a = [1, 2, 3, 4, 5];

const res = a.slice(1, 4);

console.log(res);

console.log(a)
```

Output

[2, 3, 4]

[1, 2, 3, 4, 5]

In this example

- The slice() method creates a new array by extracting elements from index 1 to 3 (exclusive of 4) from the original array.

- The original array remains unchanged, and the result is [2, 3, 4].

13. JavaScript Array some() Method

The some() method checks whether at least one of the elements of the array satisfies the condition checked by the argument function.

```
const a = [1, 2, 3, 4, 5];

let res = a.some((val) => val > 4);

console.log(res);
```

Output

```
true
```

In this example

- The some() method checks if at least one element in the array satisfies the provided condition.
- (val) => val > 4: The condition checks if any element in the array is greater than 4.
- The method stops as soon as it finds the first matching element, making it efficient for large arrays.

14. JavaScript Array map() Method

The map() method creates an array by calling a specific function on each element present in the parent array. It is a non-mutating method.

```
let a = [4, 9, 16, 25];

let sub = a.map(geeks);


function geeks() {
   return a.map(Math.sqrt);
}

console.log(sub);
```

Output

```
[ [ 2, 3, 4, 5 ], [ 2, 3, 4, 5 ], [ 2, 3, 4, 5 ], [ 2, 3, 4, 5 ] ]
```

In this example

- The code defines a geeks function, but instead of operating on the individual array 'a' elements, it applies Math.sqrt() to the entire a array, resulting in a nested array of square roots.
- The output will be an array of the same length as a, but each element will be the result of applying arr.map(Math.sqrt).

15. JavaScript Array filter() method

The filter() method in JavaScript creates a new array with all elements that pass the test implemented by the provided function. It does not modify the original array.

```
let a1 = [1, 2, 3, 4, 5]

let a2 = a1.filter((num) => num > 1)

console.log(a2)
```

Output

[2, 3, 4, 5]

In this example

- The filter() method is used to create a new array (a2) that contains all elements of the original array that satisfy the condition (num > 1).
- The filter() method does not modify the original array. It returns a new array, leaving the original one unchanged.

16. JavaScript Array reduce() Method

The reduce() method is used to reduce the array to a single value and executes a provided function for each value of the array (from left to right) and the return value of the function is stored in an accumulator.

```
let a = [88, 50, 25, 10];

let sub = a.reduce(geeks);


function geeks(tot, num) {
   return tot - num;
}

console.log(sub);
```

Output

3

In this example

- The reduce() method iterates over the array 'a' and applies the geeks function, which subtracts each element (num) from the running total (tot).
- For the array [88, 50, 25, 10], the calculation proceeds as: 88 - 50 - 25 - 10.

17. JavaScript Array reverse() method

The reverse() method is used to reverse the order of elements in an array. It modifies the array in place and returns a reference to the same array with the reversed order.

```
let a = [1, 2, 3, 4, 5];

a.reverse();

console.log(a);
```

Output

[5, 4, 3, 2, 1]

In this example

- The reverse() method reverses the order of elements in the array arr in place, modifying the original array.
- After applying reverse(), the array.

18. JavaScript Array values() method

The values() method returns a new Array Iterator object that contains the values for each index in the array.

```
const a = ["Apple", "Banana", "Cherry"];

const res = a.values();
```

```
for (const value of res) {
   console.log(value);
}
```

Output

```
Apple
Banana
Cherry
```

In this example

- The values() method returns an iterator object that allows iterating over the values of the 'a' array.
- The for...of loop is used to iterate over the iterator and log each fruit ("Apple", "Banana", "Cherry") to the console.

JavaScript Date

The JavaScript Date object represents a specific moment in time, measured in milliseconds since the Unix Epoch (January 1, 1970). It's crucial for working with date and time in web applications, providing methods for tasks like date manipulation, formatting, and calculations.

What is the JavaScript Date Object?

In JavaScript, the Date object tracks time as the number of milliseconds since the Unix Epoch. You can create a Date object using the new Date() constructor, which allows you to specify either the current date and time or a particular date.

Table of Content

Creating a Date Object

The Date object can be initialized in various ways using the new Date() constructor. It supports multiple formats for defining dates:

Syntax

```
new Date();
new Date(value);
new Date(dateString);
new Date(year, month, day, hours, minutes, seconds, milliseconds);
```

Different Ways to Create a Date Object

1. Current Date and Time

```
const currentDate = new Date();
```

2. Milliseconds since Epoch

```
const dateFromMilliseconds = new Date(1609459200000);  // Represents January 1, 2021
```

3. Date String

```
const dateFromString = new Date("March 25, 2024");
```

4. Specific Date and Time

```
const specificDate = new Date(2023, 10, 13, 5, 30, 22);  // Represents November 13, 2023, 5:30:22 AM
```

Parameters for the Date Constructor

The Date constructor allows for different parameter types to define a specific date and time. Here's a breakdown of the possible parameters:

Field	Description
value	The number of milliseconds since January 1, 1970, 00:00:00 UTC.

Field	Description
dateString	Represents a date format.
year	An integer representing the year, ranging from 1900 to 1999.
month	An integer representing the month, ranging from 0 for January to 11 for December.
day	An optional integer representing the day of the month.
hours	An optional integer representing the hour of the day.
minutes	An optional integer representing the minute of the time.
seconds	An optional integer representing the second of the time.
milliseconds	An optional integer representing the millisecond of the time.

Retrieving Date Components

You can retrieve various parts of a date using the following methods:

- getFullYear(): Returns the full year (e.g., 2024).
- getMonth(): Returns the month (0 for January, 11 for December).
- getDate(): Returns the day of the month (1-31).
- getHours(): Returns the hour (0-23).
- getMinutes(): Returns the minutes (0-59).
- getSeconds(): Returns the seconds (0-59).
- getMilliseconds(): Returns the milliseconds (0-999).

Formatting Dates

You can format dates manually or use built-in options such as Intl.DateTimeFormat to display the date in a more readable format.

```
const date = new Date();

const formattedDate = new Intl.DateTimeFormat('en-US').format(date);  // Displays as month/day/year

console.log(formattedDate);
```

Output

```
7/8/2024
```

Manipulating Dates

You can modify dates using various methods provided by the Date object, such as:

- setDate(): Adjusts the day of the month.
- setFullYear(): Sets the year.
- setMonth(): Sets the month.
- setHours(): Sets the hour.

Example 1: Adding 7 Days

The code initializes a Date object representing the current date. It then increments the date by 7 days using setDate(). Finally, it logs the modified date to the console.

```
let date = new Date();

date.setDate(date.getDate() + 7); // Adds 7 days to the current date

console.log(date);
```

Output

```
2024-07-15T10:25:17.602Z
```

Example 2: Creating a Specific Date

The code initializes a `Date` object with the provided parameters: year (1996), month (10 for November), day (13), hours (5), minutes (30), seconds (22), and milliseconds (0 by default). It then logs this date to the console.

```
// When some numbers are taken as the parameter

// then they are considered as year, month, day,

// hours, minutes, seconds, milliseconds

// respectively.

let A = new Date(1996, 10, 13, 5, 30, 22);

console.log(A);
```

Output

```
1996-11-13T05:30:22.000Z
```

JavaScript String

A JavaScript String is a sequence of characters, typically used to represent text.

- In JavaScript, there is no character type (Similar to Python and different from C, C++ and Java), so a single character string is used when we need a character.
- Like Java and Python, strings in JavaScript are immutable.

Create using Literals - Recommended

We can either use a single quote or a double quote to create a string. We can use either of the two, but it is recommended to be consistent with your choice throughout your code.

```
// Using Single Quote

let s1 = 'abcd';

console.log(s1);

// Using Double Quote

let s2 = "abcd";

console.log(s2);
```

Output

```
abcd

abcd
```

Create using Constructor

The new String() constructor creates a string object instead of a primitive string. It is generally not recommended because it can cause unexpected behavior in comparisons

```
let s = new String('abcd');

console.log(s);
```

Output

```
[String: 'abcd']
```

Template Literals (String Interpolation)

You can create strings using Template Literals. Template literals allow you to embed expressions within backticks (`) for dynamic string creation, making it more readable and versatile.

```
let s1 = 'gfg';

let s2 = `You are learning from ${s1}`;

console.log(s2);
```

Output

```
You are learning from gfg
```

Empty String

You can create an empty string by assigning either single or double quotes with no characters in between.

```
let s1 = '';
```

```
let s2 = "";

console.log(s1);

console.log(s2);
```

Output

Since the strings are empty, console.log will print two blank lines.

Multiline Strings (ES6 and later)

You can create a multiline string using backticks (``) with template literals. The backticks allows you to span the string across multiple lines, preserving the line breaks within the string.

```
let s = `
    This is a
    multiline
    string`;
console.log(s);
```

Output

```
    This is a
    multiline
    string
```

Basic Operations on JavaScript Strings

1. Finding the length of a String

You can find the length of a string using the length property.

```
let s = 'JavaScript';

let len = s.length;


console.log("String Length: " + len);
```

Output

String Length: 10

2. String Concatenation

You can combine two or more strings using + Operator.

```
let s1 = 'Java';

let s2 = 'Script';

let res = s1 + s2;


console.log("Concatenated String: " + res);
```

Output

Concatenated String: JavaScript

3. Escape Characters

We can use escape characters in string to add single quotes, dual quotes, and backslash.

\' - Inserts a single quote
\" - Inserts a double quote
\\ - Inserts a backslash

```
const s1 = "\'GfG\' is a learning portal";

const s2 = "\"GfG\" is a learning portal";

const s3 = "\\GfG\\ is a learning portal";


console.log(s1);

console.log(s2);

console.log(s3);
```

Output

'GfG' is a learning portal

"GfG" is a learning portal

\GfG\ is a learning portal

4. Breaking Long Strings

Using a backslash (\) to break a long string is not recommended, as it is not supported in strict mode. Instead, use template literals or string concatenation.

```
const s = "'GeeksforGeeks' is \

a learning portal";


console.log(s);
```

Output

'GeeksforGeeks' is a learning portal

Note: This method might not be supported on all browsers. A better way to break a string is by using the string addition.

```
const s = "'GeeksforGeeks' is a"

   + " learning portal";


console.log(s);
```

Output

'GeeksforGeeks' is a learning portal

5. Find Substring of a String

We can extract a portion of a string using the substring() method.

```
let s1 = 'JavaScript Tutorial';

let s2 = s1.substring(0, 10);


console.log(s2);
```

Output

JavaScript

6. Convert String to Uppercase and Lowercase

Convert a string to uppercase and lowercase using toUpperCase() and toLowerCase() methods.

```
let s = 'JavaScript';

let uCase = s.toUpperCase();

let lCase = s.toLowerCase();


console.log(uCase);

console.log(lCase);
```

Output

JAVASCRIPT

javascript

7. String Search in JavaScript

Find the first index of a substring within a string using indexOf() method.

```
let s1 = 'def abc abc';

let i = s1.indexOf('abc');


console.log(i);
```

Output

4

8. String Replace in JavaScript

Replace occurrences of a substring using the replace() method. By default, replace() only replaces the first occurrence. To replace all occurrences, use a regular expression with the g flag.

```
let s1 = 'Learn HTML at GfG and HTML is useful';
let s2 = s1.replace(/HTML/g, 'JavaScript');

console.log(s2);
```

Output

```
Learn JavaScript at GfG and JavaScript is useful
```

9. Trimming Whitespace from String

Remove leading and trailing whitespaces using trim() method.

```
let s1 = '    Learn JavaScript      ';
let s2 = s1.trim();

console.log(s2);
```

Output

```
Learn JavaScript
```

10. Access Characters from String

Access individual characters in a string using bracket notation and charAt() method.

```
let s1 = 'Learn JavaScript';
let s2 = s1[6];
console.log(s2);

s2 = s1.charAt(6);
console.log(s2);
```

Output

```
J
J
```

11. String Comparison in JavaScript

There are some inbuilt methods that can be used to compare strings such as the equality operator and another like localeCompare() method.

```
let s1 = "Ajay"
let s2 = new String("Ajay");

console.log(s1 == s2);  // true (type coercion)
console.log(s1 === s2); // false (strict comparison)
```

```
console.log(s1.localeCompare(s2)); // 0 (means they are equal lexicographically)
```

Output

```
true

false

0
```

Note: The equality operator (==) may return true when comparing a string object with a primitive string due to type coercion. However, === (strict equality) returns false because objects and primitives are different types. The localeCompare() method compares strings lexicographically.

12. Passing JavaScript String as Objects

We can create a JavaScript string using the new keyword.

```
const str = new String("GeeksforGeeks");


console.log(str);
```

Output

```
[String: 'GeeksforGeeks']
```

String Methods

JavaScript provides a variety of built-in methods to work with strings — allowing you to extract, modify, search, and format text with ease. Below are some of the most commonly used core string methods:

slice()

slice() extracts a part of the string based on the given stating-index and ending-index and returns a new string.

```
// Define a string variable

let A = 'Geeks for Geeks';


// Use the slice() method to extract a substring

let b = A.slice(0, 5);

let c = A.slice(6, 9);

let d = A.slice(10);


// Output the value of variable

console.log(b);

console.log(c);

console.log(d);
```

Output

```
Geeks

for

Geeks
```

substring()

substring() returns the part of the given string from the start index to the end index. Indexing starts from zero (0).

```
// Define a string variable

let str = "Mind, Power, Soul";


// Use the substring() method to extract a substring

let part = str.substring(6, 11);


// Output the value of variable

console.log(part);
```

Output

Power

substr()

substr() This method returns the specified number of characters from the specified index from the given string. It extracts a part of the original string.

```
// Define a string variable 'str'

let str = "Mind, Power, Soul";


// Use the substr() method to extract a substring f

let part = str.substr(6, 5);


// Output the value of variable

console.log(part);
```

Output

Power

replace()

replace() replaces a part of the given string with another string or a regular expression. The original string will remain unchanged.

```
// Define a string variable 'str'

let str = "Mind, Power, Soul";


// Use the replace() method to replace the substring

let part = str.replace("Power", "Space");


// Output the resulting string after replacement

console.log(part);
```

Output

Mind, Space, Soul

replaceAll()

replaceAll() returns a new string after replacing all the matches of a string with a specified string or a regular expression. The original string is left unchanged after this operation.

```
// Define a string variable 'str'
let str = "Mind, Power, Power, Soul";

// Use the replaceAll() method to replace all occurrences
//of "Power" with "Space" in the string 'str'
let part = str.replaceAll("Power", "Space");

// Output the resulting string after replacement
console.log(part);
```

Output

Mind, Space, Space, Soul

toUpperCase()

toUpperCase() converts all the characters present in the String to upper case and returns a new String with all characters in upper case. This method accepts single parameter **stringVariable** string that you want to convert in upper case.

```
// Define a string variable
let gfg = 'GFG ';

// Define another string variable
let geeks = 'stands-for-GeeksforGeeks';

// Convert the string 'geeks' to uppercase using the toUpperCase() method
console.log(geeks.toUpperCase());
```

Output

STANDS-FOR-GEEKSFORGEEKS

toLowerCase()

toLowerCase() converts all the characters present in the so lowercase and returns a new string with all the characters in lowercase.

```
// Define a string variable
let gfg = 'GFG ';

// Define a string variable
```

```
let geeks = 'stands-for-GeeksforGeeks';


// Convert the string 'geeks' to lowercase using the toLowerCase() method
console.log(geeks.toLowerCase());
```

Output

```
stands-for-geeksforgeeks
```

concat()

concat() combines the text of two strings and returns a new combined or joined string. To concatenate two strings, we use the **concat()** method on one object of string and send another object of string as a parameter. This method accepts one argument. The variable contains text in double quotes or single quotes.

```
let gfg = 'GFG ';
let geeks = 'stands for GeeksforGeeks';


// Accessing concat method on an object
// of String passing another object
// as a parameter
console.log(gfg.concat(geeks));
```

Output

```
GFG stands for GeeksforGeeks
```

trim()

trim() is used to remove either white spaces from the given string. This method returns a new string with removed white spaces. This method is called on a String object. This method doesn't accept any parameter.

```
let gfg = 'GFG    ';
let geeks = 'stands-for-GeeksforGeeks';


// Storing new object of string
// with removed white spaces
let newGfg = gfg.trim();


// Old length
console.log(gfg.length);
```

```
// New length
console.log(newGfg.length)
```

Output

7

3

trimStart()

trimStart() removes whitespace from the beginning of a string. The value of the string is not modified in any manner, including any whitespace present after the string.

```
// Define a string variable
let str = "  Soul";

// Output the original value of the string
console.log(str);

// Use the trimStart() method to remove leading whitespace from the string 'str'
let part = str.trimStart();

// Output the resulting string after removing leading whitespace
console.log(part);
```

Output

```
  Soul
Soul
```

trimEnd()

trimEnd() removes white space from the end of a string. The value of the string is not modified in any manner, including any white-space present before the string.

```
// Define a string variable
let str = "Soul  ";

// Output the original value of the string 'str'
console.log(str);
```

```
// Use the trimEnd() method to remove trailing whitespace from the string 'str'
let part = str.trimEnd();

// Output the resulting string after removing trailing whitespace
console.log(part);
```

Output

```
Soul
Soul
```

padStart()

padStart() pad a string with another string until it reaches the given length. The padding is applied from the left end of the string.

```
// Define a string variable
let stone = "Soul";

// Use the padStart() method to add padding characters "Mind "
//to the beginning of the string 'stone'
stone = stone.padStart(9, "Mind ");

// Output the resulting string after padding
console.log(stone);
```

Output

```
Mind Soul
```

padEnd()

padEnd() pad a string with another string until it reaches the given length. The padding is applied from the right end of the string.

```
// Define a string variable
let stone = "Soul";

// Use the padEnd() method to add padding characters
//" Power" to the end of the string 'stone'
stone = stone.padEnd(10, " Power");
```

```
// Output the resulting string after padding
console.log(stone);
```

Output

```
Soul Power
```

charAt()

charAt() returns the character at the specified index. String in JavaScript has zero-based indexing.

```
let gfg = 'GeeksforGeeks';
let geeks = 'GfG is the best platform to learn and\n'+
'experience Computer Science.';

// Print the string as it is
console.log(gfg);

console.log(geeks);

// As string index starts from zero
// It will return first character of string
console.log(gfg.charAt(0));

console.log(geeks.charAt(5));
```

Output

```
GeeksforGeeks
GfG is the best platform to learn and
experience Computer Science.
G
s
```

charCodeAt()

charCodeAt() returns a number that represents the **Unicode** value of the character at the *specified index*. This method accepts one argument.

```
let gfg = 'GeeksforGeeks';
let geeks = 'GfG is the best platform\n\
```

```
to learn and experience\n\
Computer Science.';

// Return a number indicating Unicode
// value of character at index 0 ('G')
console.log(gfg.charCodeAt(0));
console.log(geeks.charCodeAt(5));
```

Output

```
71
115
```

split()

split() splits the string into an array of sub-strings. This method returns an array. This method accepts a single parameter **character** on which you want to split the string.

```
let gfg = 'GFG '
let geeks = 'stands-for-GeeksforGeeks'

// Split string on '-'.
console.log(geeks.split('-'))
```

Output

```
[ 'stands', 'for', 'GeeksforGeeks' ]
```

More JS String Methods

Below is the JavaScript string functions list:

Instance Methods	Description
at()	Find the character at the specified index.
anchor()	Creates an anchor element that is used as a hypertext target.

Instance Methods	Description
charAt()	Returns that character at the given index of the string.
charCodeAt()	Returns a Unicode character set code unit of the character present at the index in the string.
codePointAt()	Return a non-negative integer value i.e, the code point value of the specified element.
concat()	Join two or more strings together in JavaScript.
endsWith()	Whether the given string ends with the characters of the specified string or not.
includes()	Returns true if the string contains the characters, otherwise, it returns false.
indexOf()	Finds the index of the first occurrence of the argument string in the given string.
lastIndexOf()	Finds the index of the last occurrence of the argument string in the given string.
localeCompare()	Compare any two elements and returns a positive number
match()	Search a string for a match against any regular expression.
matchAll()	Return all the iterators matching the reference string against a regular expression.

Instance Methods	Description
normalize()	Return a Unicode normalization form of a given input string.
padEnd()	Pad a string with another string until it reaches the given length from rightend.
padStart()	Pad a string with another string until it reaches the given length from leftend.
repeat()	Build a new string containing a specified number of copies of the string.
replace()	Replace a part of the given string with some another string or a regular expression
replaceAll()	Returns a new string after replacing all the matches of a string with a specified string/regex.
search()	Search for a match in between regular expressions and a given string object.
slice()	Return a part or slice of the given input string.
split()	Separate given string into substrings using a specified separator provided in the argument.
startsWith()	Check whether the given string starts with the characters of the specified string or not.
substr()	Returns the specified number of characters from the specified index from the given string.

Instance Methods	Description
substring()	Return the part of the given string from the start index to the end index.
toLowerCase()	Converts the entire string to lowercase.
toLocaleLowerCase()	Returns the calling string value converted to a lowercase letter.
toLocaleUpperCase()	Returns the calling string value converted to a uppercase letter.
toUpperCase()	Converts the entire string to uppercase.
toString()	Return the given string itself.
trim()	Remove the white spaces from both ends of the given string.
trimEnd()	Remove white space from the end of a string.
trimStart()	Remove white space from the start of a string.
valueOf()	Return the value of the given string.
string[Symbol.iterator]()	This method is used to make String iterable. [@@iterator]() returns an iterator object which iterates over all code points of the String.
fromCharCode(n1, n2, ..., nX)	This method is used to create a string from the given sequence of UTF-16 code units. This method returns a string, not a string object.

Instance Methods	Description
fromCodePoint(a1, a2, a3, ...)	This method in JavaScript that is used to return a string or an element for the given sequence of code point values (ASCII value).
isWellFormed()	This method is used to check if the string contains a lone surrogate or not
String.raw(str, ...sub)	This is a static method that is used to get the raw string form of template literals. These strings do not process escape characters.
toWellFormed()	This method is used to return where all lone surrogates of this string are replaced with the Unicode replacement character.